

Contents

1. How to Organize the Data	2
2. Add a Vertical Topology Group	3
3. Reproject Only the Working Layer	5
4. Split the Layer by topology_group	6
5. Split Lines Within Each Group.....	6
6. Merge the Processed Groups	9
7. What Can Be Safely Optimized	9
8. Create New gid Values.....	10
9. Reproject the Final Layer Back to EPSG:4326	10
10. Do Not Include All OSM Columns in the Final Upload	11
11. Upload the Result as a New Table	12
12. Add Routing Columns and Weights	13
If the Previous Query Already Created Columns with Incorrect Defaults.....	14
13. Calculate the Base Cost	15
14. Create Vertices and source/target.....	15
15. Load Each Source One by One and Calculate Weights	17
16. What Can Be Calculated from the Network's OSM Fields	18
Stairs	18
Tactile paving.....	18
17. SQL Patterns for External Sources	23
Spatial Update for light_weight.....	25
What to Upload to Supabase	26
SQL for Debug Calculation Before the Update	29
POI/amenities.....	34
18. Process WMS and Rasters Separately	41
Noise	41
Traffic Lights	48
19. Final Check	58

1. How to Organize the Data

Save two local layers using QuickOSM.

The first is the full archive layer: `muc_ways_master`. It keeps all original OSM fields, the geometry, and the old `gid`. Do not edit it further.

The second is a lightweight working layer: `muc_ways_topology_raw`.

It should contain only:

```
parent_gid
geometry_source
highway
layer
bridge
tunnel
level
indoor
location
```

All other fields remain in `muc_ways_master`. The link is preserved through:
`muc_ways_topology.parent_gid → muc_ways_master.gid`

This will significantly speed up local operations: QGIS will not need to carry hundreds of text-based OSM fields through every split operation.

To create the lightweight topology layer in QGIS:

```
Processing
→ Toolbox
→ Refactor fields
```

Parameters:

```
Input layer: muc_ways_master
Output: Save to GeoPackage...
Layer name: muc_ways_topology_raw
```

Rename the original `gid` field:

```
gid → parent_gid
Type: Integer64
```

Be sure to keep the geometry and the fields listed above. The full layer with all OSM attributes remains separate, so no data is lost.

2. Add a Vertical Topology Group

In muc_ways_topology_raw, open:

Open Attribute Table → Field Calculator

Create a field:

Field name: topology_group

Field type: Text (string)

Length: 150

Expression:

```
with_variable(
    'layer_num',
    coalesce(
        to_int(
            regexp_substr(
                trim(to_string("layer")),
                '^-[0-9]+'
            )
        ),
        0
    ),

    with_variable(
        'level_num',
        coalesce(
            regexp_substr(
                trim(to_string("level")),
                '-?[0-9]+([.,][0-9]+)?'
            ),
            '0'
        ),
        ),

    with_variable(
        'bridge_yes',
        lower(
            trim(
                coalesce(to_string("bridge"), '')
            )
        ) NOT IN ('', 'no', 'false', '0'),

    with_variable(
        'tunnel_yes',
        lower(
            trim(
                coalesce(to_string("tunnel"), '')
            )
        ) NOT IN ('', 'no', 'false', '0')
    )
```

```

    )
    ) NOT IN ('', 'no', 'false', '0'),

with_variable(
    'indoor_yes',
    lower(
        trim(
            coalesce(to_string("indoor"), '')
        )
    ) IN ('yes', 'true', '1'),

with_variable(
    'location_norm',
    lower(
        trim(
            coalesce(to_string("location"), '')
        )
    ),

CASE
    WHEN @indoor_yes
    THEN
        'indoor_level=' ||
        replace(@level_num, ',', '.')

    WHEN @bridge_yes
    THEN
        'bridge_layer=' ||
        to_string(
            CASE
                WHEN @layer_num = 0 THEN 1
                ELSE @layer_num
            END
        )

    WHEN @tunnel_yes
        OR @location_norm IN (
            'underground',
            'underwater'
        )
    THEN
        'tunnel_layer=' ||
        to_string(
            CASE
                WHEN @layer_num = 0 THEN -1
                ELSE @layer_num
            END
        )

    WHEN @location_norm = 'roof'
    THEN

```

```
'roof'

    WHEN @layer_num <> 0
    THEN
        'other_layer=' ||
        to_string(@layer_num)

    ELSE
        'surface'
    END
END
)
```

The logic is as follows:

```
regular surface streets → one group
bridges                → another group
tunnels                → another group
different indoor levels → different groups
```

Lines from different groups must not split each other at ordinary 2D intersections. At the same time, a connecting ramp from another level can still connect to the network if its endpoint geometrically matches the endpoint of another edge.

3. Reproject Only the Working Layer

Run:

Processing

→ Toolbox

→ Reproject layer

Parameters:

Input layer: muc_ways_topology_raw

Target CRS: EPSG:25832

Output layer: muc_ways_topology_25832

This is an actual coordinate transformation. The source layer in EPSG:4326 remains unchanged. Processing in EPSG:25832 is preferred because distances and tolerances are specified in meters.

4. Split the Layer by topology_group

Find:

Processing

→ Toolbox

→ Split vector layer

Parameters:

Input layer: muc_ways_topology_25832

Unique ID field: topology_group

Output directory: a separate temporary folder (for example:

C:\Users\userr\Documents\CAT\muc_topology_groups)

QGIS will create a separate line layer for each level. This is preferable to splitting the entire network at once: bridges, tunnels, and surface roads will not split one another.

5. Split Lines Within Each Group

The best tool for this operation is GRASS:

Processing Toolbox → GRASS → v.clean

If the GRASS tools are not visible:

Settings → Options → Processing → Providers → GRASS → Activate

You can run v.clean for all files through:

Right-click v.clean → Execute as Batch Process

1. In the Layer to clean column, click the top row:
Autofill... → Select from Open Layers...

Select all required layers using Ctrl + A or Shift, then click OK.

If the 239 layers are stored as separate files in one folder, it is easier to use:

Autofill... → Add All Files from a Directory...

After you select them, QGIS will add all required rows automatically.

2. In the Input feature type column, you may currently see something like:
centroid, area, face, kernel

This is not correct for this network. Select only:

line

Set this in the first working row, then use the top row of that column:

Autofill... → Fill Down

3. In Cleaning tool, only the following may currently be selected:

break

Select two tools: break,rmdupl

Then again:

Autofill... → Fill Down

break splits lines at intersections, and rmdupl removes exact duplicates created after the split.

4. In the Threshold column, enter: 0,0

Then run: Autofill... → Fill Down

For break and rmdupl, the threshold is effectively not used, but when two tools are selected the interface still requires two values. Zeros are suitable.

5. In the Cleaned column, do not create 239 temporary layers in memory. Create a separate folder, for example:

C:\Users\userr\Documents\CAT\muc_cleaned_groups

In the first working row, click ..., select Save to File..., and save the first result as: cleaned_001.gpkg

In the automatic fill dialog that appears, select: Fill with numbers

QGIS will generate separate names for all subsequent outputs. The Batch interface supports automatic output-name generation and filling values from input parameters or numbers.

In the Batch Processing table, add all created topology-group layers.

After a test run on one group, open the result's Attribute Table.

The parent_gid field must be preserved. New parts of the same source line must receive the same parent_gid.

For example, parent_gid = 1007 may appear in three rows after splitting:

1007 – part A

1007 – part B

1007 – part C

`Split with lines` splits input lines where they intersect the lines in the split layer and is also suitable for this task. However, it performs only the split, while `v.clean` is a broader topological-cleaning tool. `Split with lines` is usually slower than `v.clean`, but it preserves input feature attributes more predictably.

`Split with lines` takes one layer as the `Input layer` and another as the `Split layer`, then splits the input lines at every geometric intersection. If the same layer is specified in both fields, the lines will act as cutting lines for one another. QGIS preserves the attributes of the input features, but recreates internal FIDs, and the result may contain multipart geometries.

For a layer where `v.clean` hangs, use the following alternative:

`Split with lines`

→ `Multipart to singleparts`

→ `Delete duplicate geometries`

This produces an almost equivalent result to the `break,rmdupl` sequence.

`Split with lines` settings:

`Input layer: topology_group_tunnel_layer=1`

`Split layer: topology_group_tunnel_layer=1`

`Output: tunnel_layer_1_split.gpkg`

You can select the same layer in both fields. If QGIS does not allow the same reference to be selected twice, add the source GeoPackage to the project a second time and use the second copy as the `Split layer`.

After it finishes, be sure to run:

`Processing Toolbox`

→ `Multipart to singleparts`

QGIS documentation explicitly notes that split features may remain multipart. For `pgRouting`, you need individual `LineString` geometries, not `MultiLineString`.

Then, because processing is performed within a single topology group, you can safely run:

`Processing Toolbox` → `Delete duplicate geometries`

This replaces rmdupl. Within a single group, bridges and surface features are already separated, so removing exact geometric duplicates should not accidentally remove a feature from another vertical level.

After that, it is also recommended to run: Processing Toolbox → Check validity.

6. Merge the Processed Groups

After processing all groups:

Processing

→ Toolbox

→ Merge vector layers

Select all v.clean results.

Parameters:

Destination CRS: EPSG:25832

Output: muc_ways_noded_25832

Then:

Processing

→ Toolbox

→ Multipart to singleparts

The muc_ways_noded_single_25832 result should become the basis of the graph.

Do not run a global Delete duplicate geometries after merging the groups.

Geometrically coincident lines may belong to different levels, for example a tunnel and the surface.

7. What Can Be Safely Optimized

You can run Remove duplicate vertices. It removes repeated consecutive vertices within a single geometry without merging multiple edges into one.

For now, do not run:

Dissolve and Merge lines can join several graph edges and remove an intersection node.

Simplify can slightly move the geometry and break the exact endpoint match between neighboring lines.

Promote to multipart will create MultiLineString geometry, while the production table should contain LineString.

There is no need to simplify the main routing geometry. The backend can already apply ST_SimplifyPreserveTopology to the geometry returned to the client in simplified mode, while the source graph can remain exact.

8. Create New gid Values

After all splitting operations:

Processing

→ Toolbox

→ Add autoincremental field

Parameters:

Field name: gid

Start value: 1

The old identifier remains as parent_gid. The new gid identifies a specific routing edge. After this, the geometry must not be split or merged without regenerating gid values.

9. Reproject the Final Layer Back to EPSG:4326

Processing

→ Toolbox

→ Reproject layer

Parameters:

Input: muc_ways_noded_single_25832

Target CRS: EPSG:4326

Output: muc_ways_noded_4326

Run this check first:

```
SELECT
  ST_SRID(geom) AS declared_srid,
  GeometryType(geom) AS geometry_type,
  COUNT(*) AS feature_count,
  ST_Extent(geom) AS coordinate_extent
FROM public.muc_ways_noded_4326
GROUP BY
  ST_SRID(geom),
  GeometryType(geom);
```

For a correct EPSG:4326 layer, the result should look approximately like this:

`BOX(11.3 48.0, 11.8 48.4).`

If you see something like `BOX(650000 5320000, 710000 5360000)`, the geometry has actually remained in EPSG:25832 and was only declared as EPSG:4326.

10. Do Not Include All OSM Columns in the Final Upload

Keep the complete set locally in: `muc_ways_master`

Reduce the final server layer through:

Processing → Toolbox → Refactor fields

Keep:

`gid`

`parent_gid`

`source`

`target`

`cost`

`reverse_cost`

`noise_weight`

`light_weight`

`trafficlight_weight`

`tactile_weight`

`tree_weight`

`temp_weight_s`

`temp_weight_w`

`blue_weight`

`green_weight`

`station_weight`

`wc_d_weight`

`path_width_weight`

`stair_weight`

`obstacle_weight`

`slope_weight`

`uneven_surfaces_weight`

`poor_pavement_weight`

`kerbs_h_weight`

`facilities_weight`

```
pedestrian_flow_weight
geometry
```

source, target, cost, and reverse_cost can be created later in PostGIS. This makes repeated uploads much faster because instead of hundreds of OSM columns, you transfer only about 25-30 backend fields that are actually needed.

11. Upload the Result as a New Table

You can open DB Manager and continue working in the main QGIS window.

Right-click public, choose Import Vector Layer..., and select the open muc_ways_final layer. Alternatively, drag muc_ways_final from the Layers panel directly onto the public schema. QGIS officially supports both methods.

Check after upload:

```
SELECT
  COUNT(*) AS total_edges,
  COUNT(gid) AS gid_not_null,
  COUNT(DISTINCT gid) AS unique_gid
FROM public.muc_ways_noded_4326;
```

All three values should match.

Geometry check:

```
SELECT
  ST_SRID(geom) AS srid,
  GeometryType(geom) AS geometry_type,
  COUNT(*) AS feature_count
FROM public.muc_ways_noded_4326
GROUP BY
  ST_SRID(geom),
  GeometryType(geom);
```

Expected result:

SRID: 4326

Geometry type: LINESTRING

Problem check:

```
SELECT
  COUNT(*) FILTER (WHERE the_geom IS NULL) AS null_geometry,
```

```
COUNT(*) FILTER (WHERE ST_IsEmpty(the_geom)) AS empty_geometry,  
COUNT(*) FILTER (WHERE NOT ST_IsValid(the_geom)) AS invalid_geometry,  
COUNT(*) FILTER (WHERE ST_NPoints(the_geom) < 2) AS too_few_points  
FROM public.muc_ways_noded_4326;
```

All values should be 0.

12. Add Routing Columns and Weights

```
ALTER TABLE public.muc_ways_noded_4326  
  ADD COLUMN IF NOT EXISTS source bigint,  
  ADD COLUMN IF NOT EXISTS target bigint,  
  ADD COLUMN IF NOT EXISTS cost double precision,  
  ADD COLUMN IF NOT EXISTS reverse_cost double precision,  
  
  -- Negative factors: discomfort is active when value = 1  
  ADD COLUMN IF NOT EXISTS noise_weight smallint NOT NULL DEFAULT 0,  
  ADD COLUMN IF NOT EXISTS trafficlight_weight smallint NOT NULL DEFAULT 0,  
  ADD COLUMN IF NOT EXISTS temp_weight_s integer NOT NULL DEFAULT 0,  
  ADD COLUMN IF NOT EXISTS temp_weight_w smallint NOT NULL DEFAULT 0,  
  ADD COLUMN IF NOT EXISTS path_width_weight smallint NOT NULL DEFAULT 0,  
  ADD COLUMN IF NOT EXISTS stair_weight smallint NOT NULL DEFAULT 0,  
  ADD COLUMN IF NOT EXISTS obstacle_weight smallint NOT NULL DEFAULT 0,  
  ADD COLUMN IF NOT EXISTS slope_weight smallint NOT NULL DEFAULT 0,  
  ADD COLUMN IF NOT EXISTS uneven_surfaces_weight smallint NOT NULL DEFAULT 0,  
  ADD COLUMN IF NOT EXISTS poor_pavement_weight smallint NOT NULL DEFAULT 0,  
  ADD COLUMN IF NOT EXISTS kerbs_h_weight smallint NOT NULL DEFAULT 0,  
  ADD COLUMN IF NOT EXISTS pedestrian_flow_weight smallint NOT NULL DEFAULT 0,  
  
  -- Positive factors: discomfort is active when value = 0  
  ADD COLUMN IF NOT EXISTS light_weight smallint NOT NULL DEFAULT 1,  
  ADD COLUMN IF NOT EXISTS tactile_weight smallint NOT NULL DEFAULT 1,  
  ADD COLUMN IF NOT EXISTS tree_weight smallint NOT NULL DEFAULT 1,  
  ADD COLUMN IF NOT EXISTS blue_weight smallint NOT NULL DEFAULT 1,  
  ADD COLUMN IF NOT EXISTS green_weight smallint NOT NULL DEFAULT 1,  
  ADD COLUMN IF NOT EXISTS station_weight smallint NOT NULL DEFAULT 1,  
  ADD COLUMN IF NOT EXISTS wc_d_weight smallint NOT NULL DEFAULT 1,  
  ADD COLUMN IF NOT EXISTS facilities_weight smallint NOT NULL DEFAULT 1;
```

Pay attention to the exact names:

```
trafficlight_weight  
uneven_surfaces_weight  
facilities_weight  
kerbs_h_weight  
wc_d_weight  
path_width_weight
```

Do not use variants such as:

trafficLight_weight
uneven_surface_weight
facility_weight
kerb_weight
wc_disabled_weight
narrow_roads_weight

If the Previous Query Already Created Columns with Incorrect Defaults

ADD COLUMN IF NOT EXISTS does not modify columns that already exist. In that case, if you have not yet calculated the actual weights, run:

```
ALTER TABLE public.muc_ways_noded_4326
  ALTER COLUMN noise_weight SET DEFAULT 0,
  ALTER COLUMN trafficlight_weight SET DEFAULT 0,
  ALTER COLUMN temp_weight_s SET DEFAULT 0,
  ALTER COLUMN temp_weight_w SET DEFAULT 0,
  ALTER COLUMN path_width_weight SET DEFAULT 0,
  ALTER COLUMN stair_weight SET DEFAULT 0,
  ALTER COLUMN obstacle_weight SET DEFAULT 0,
  ALTER COLUMN slope_weight SET DEFAULT 0,
  ALTER COLUMN uneven_surfaces_weight SET DEFAULT 0,
  ALTER COLUMN poor_pavement_weight SET DEFAULT 0,
  ALTER COLUMN kerbs_h_weight SET DEFAULT 0,
  ALTER COLUMN pedestrian_flow_weight SET DEFAULT 0,

  ALTER COLUMN light_weight SET DEFAULT 1,
  ALTER COLUMN tactile_weight SET DEFAULT 1,
  ALTER COLUMN tree_weight SET DEFAULT 1,
  ALTER COLUMN blue_weight SET DEFAULT 1,
  ALTER COLUMN green_weight SET DEFAULT 1,
  ALTER COLUMN station_weight SET DEFAULT 1,
  ALTER COLUMN wc_d_weight SET DEFAULT 1,
  ALTER COLUMN facilities_weight SET DEFAULT 1;
```

After running it, you can check the structure:

```
SELECT
  column_name,
  data_type,
  column_default,
  is_nullable
FROM information_schema.columns
WHERE
  table_schema = 'staging'
  AND table_name = 'muc_ways_work'
  AND column_name LIKE '%weight%'
ORDER BY ordinal_position;
```

13. Calculate the Base Cost

The geometry is stored in EPSG:4326, so a regular ST_Length(the_geom) returns degrees. To get meters, use geography:

```
UPDATE public.muc_ways_noded_4326

SET
    cost = ST_Length(geom::geography),
    reverse_cost = ST_Length(geom::geography);
```

Check:

```
SELECT
    MIN(cost) AS min_cost,
    AVG(cost) AS avg_cost,
    MAX(cost) AS max_cost,
    COUNT(*) FILTER (WHERE cost <= 0) AS invalid_cost
FROM public.muc_ways_noded_4326;
```

14. Create Vertices and source/target

Newer versions of pgRouting moved away from the old approach in which pgr_createTopology created tables and indexes automatically. The function was deprecated in pgRouting 3.8 and removed in 4.0; the newer approach leaves vertex and column creation under developer control.

Use clear SQL that does not depend on the pgr_createTopology version.

```
DROP TABLE IF EXISTS public.muc_ways_vertices_pgr;

CREATE TABLE public.muc_ways_vertices_pgr AS
WITH endpoints AS (
    SELECT
        ST_SnapToGrid(
            ST_StartPoint(geom),
            0.00000001
        )::geometry(Point, 4326) AS geom
    FROM public.muc_ways_noded_4326

    UNION

    SELECT
        ST_SnapToGrid(
            ST_EndPoint(geom),
            0.00000001
        )::geometry(Point, 4326) AS geom
    FROM public.muc_ways_noded_4326
)
SELECT
```

```
ROW_NUMBER() OVER (  
    ORDER BY ST_X(geom), ST_Y(geom)  
)::bigint AS id,  
geom  
FROM endpoints;
```

Add the key and index:

```
ALTER TABLE public.muc_ways_vertices_pgr  
ADD CONSTRAINT muc_ways_vertices_pgr_pkey PRIMARY KEY (id);  
  
CREATE INDEX muc_ways_vertices_pgr_geom_idx  
ON public.muc_ways_vertices_pgr  
USING GIST (the_geom);
```

Populate source:

```
UPDATE public.muc_ways_noded_4326 AS w  
SET source = v.id  
FROM public.muc_ways_vertices_pgr AS v  
WHERE  
    ST_SnapToGrid(  
        ST_StartPoint(w.geom),  
        0.00000001  
    ) = v.geom;
```

Populate target:

```
UPDATE public.muc_ways_work AS w  
SET target = v.id  
FROM public.muc_ways_vertices_pgr AS v  
WHERE  
    ST_SnapToGrid(  
        ST_EndPoint(w.the_geom),  
        0.00000001  
    ) = v.the_geom;
```

Check:

```
SELECT  
    COUNT(*) AS total_edges,  
    COUNT(*) FILTER (  
        WHERE source IS NULL OR target IS NULL  
    ) AS missing_vertices,  
    COUNT(*) FILTER (  
        WHERE source = target  
    ) AS loops  
FROM public.muc_ways_work;
```

missing_vertices should be 0.

Important: this SQL connects matching line endpoints. It does not create a connection where one line crosses the middle of another without being split. Therefore, the QGIS splitting at real intersections must be completed before this step.

15. Load Each Source One by One and Calculate Weights

Do not run one huge UPDATE ... CASE ... statement immediately without a way to verify the result. For each factor, create a table containing the gid values of the matching edges.

Example for lighting:

```
DROP TABLE IF EXISTS staging.muc_light_candidates;

CREATE TABLE staging.muc_light_candidates AS
SELECT DISTINCT
    w.gid
FROM staging.muc_ways_work AS w
JOIN staging.muc_street_lamps AS lamp
    ON ST_DWithin(
        w.the_geom::geography,
        lamp.geom::geography,
        20
    );
```

Check:

```
SELECT COUNT(*)
FROM staging.muc_light_candidates;
```

Then:

```
UPDATE staging.muc_ways_work
SET light_weight = 0;

UPDATE staging.muc_ways_work AS w
SET light_weight = 1
FROM staging.muc_light_candidates AS candidate
WHERE candidate.gid = w.gid;
```

The advantage of this approach is that the candidate layer can be opened from PostGIS in QGIS and checked visually before updating the production table.

Do not use the same 20 m buffer for all factors. The instruction uses it as a general example, but:

- 20 m may be reasonable for street lamps;
- 20 m is too much for tactile paving;
- stops and toilets require significantly larger radii;
- noise requires a level polygon rather than a buffer.

The exact values should be defined in a separate documented project policy.

16. What Can Be Calculated from the Network's OSM Fields

These weights can be calculated directly in public.muc_ways_noded_4326 if you kept the relevant columns. Alternatively, you can use QuickOSM in QGIS to download the required layers and populate the corresponding table fields through SQL in Supabase.

Stairs

```
UPDATE public.muc_ways_noded_4326
SET stair_weight =
  CASE
    WHEN LOWER(COALESCE(highway, '')) = 'steps'
      AND LOWER(COALESCE(ramp, '')) NOT IN (
        'yes',
        'wheelchair',
        'stroller'
      )
      AND LOWER(COALESCE("ramp:wheelchair", '')) <> 'yes'
    THEN 1
    ELSE 0
  END;
```

This represents a “stairs only” indicator, not simply the presence of any stairs.

Tactile paving

You can download the relevant layers through QuickOSM in QGIS.

1. Download Tactile Paving with QuickOSM

In QGIS, open:

Vector → QuickOSM → QuickOSM...

QuickOSM works through the Overpass API and lets you specify an OSM key/value directly.

On the Quick query tab, set:

Key: tactile_paving

Value: yes

For the query area, select the current city's area. For example, if the Munich boundary is already loaded, it is better to use it as the query area instead of a very large Canvas Extent.

For example:

Key	tactile_paving
Value	yes

In Layer extent / Canvas extent
and click Run query.

Do not simply set tactile_paving = * if you specifically need confirmed tactile paving.
OSM also contains values such as:

```
tactile_paving=no  
tactile_paving=incorrect  
tactile_paving=separate
```

For a positive tactile_weight, the main value we need is yes.

OSM allows tactile_paving=* to be used on node, way, and area objects.

For example:

- point — tactile paving at a crossing, kerb, or stop;
- line — a tactile guidance strip along a path;
- polygon — tactile paving mapped as a separate area or part of a platform.

At pedestrian crossings, tactile paving is very often stored on a node together with crossing/kerb information.

Therefore, do not discard the point layer just because tactile paving may visually look like an “area of paving tiles.”

2. After QuickOSM, Save the Three Geometry Types and Upload Them to Supabase

OSM allows tactile_paving=* to be used on node, way, and area objects.

For example:

- point — tactile paving at a crossing, kerb, or stop;
- line — a tactile guidance strip along a path;
- polygon — tactile paving mapped as a separate area or part of a platform.

At pedestrian crossings, tactile paving is very often stored on a node together with crossing/kerb information. Depending on the data that is actually available, several layers may appear in the Layers panel after the query.

Save each non-empty layer separately:

right-click → Export → Save Features As...

```
munich_tactile_points.geojson  
munich_tactile_lines.geojson  
munich_tactile_polygons.geojson
```

CRS must be: EPSG:4326 — WGS 84. Upload the layers to Supabase/PostGIS.

Use unique city-specific table names, for example for Munich. This is better than using a generic name such as `tactile_lines`, because once several cities are added, the layers will remain easy to distinguish.

3. Check the Uploaded Tables

In the Supabase SQL Editor:

```
SELECT ST_SRID(geom)
FROM munich_tactile_points
LIMIT 5;

SELECT ST_SRID(geom)
FROM munich_tactile_lines
LIMIT 5;

SELECT ST_SRID(geom)
FROM munich_tactile_polygons
LIMIT 5;
```

Expected result: 4326

4. Add Spatial Indexes

```
CREATE INDEX IF NOT EXISTS munich_tactile_points_geom_gix
ON munich_tactile_points
USING GIST (geom);

CREATE INDEX IF NOT EXISTS munich_tactile_lines_geom_gix
ON munich_tactile_lines
USING GIST (geom);

CREATE INDEX IF NOT EXISTS munich_tactile_polygons_geom_gix
ON munich_tactile_polygons
USING GIST (geom);
```

5. Most Important: Include Points + Lines + Polygons

It is better to perform the calculation in several short steps rather than one large CASE + 3 EXISTS statement.

First, create geography indexes. This only needs to be done once:

```
CREATE INDEX IF NOT EXISTS muc_ways_noded_4326_v2_geog_gix
ON muc_ways_noded_4326_v2
USING GIST ((geom::geography));

CREATE INDEX IF NOT EXISTS muc_tactile_points_geog_gix
ON muc_tactile_points_tmp_clean
USING GIST ((geom::geography));

CREATE INDEX IF NOT EXISTS muc_tactile_lines_geog_gix
ON muc_tactile_lines_tmp_clean
```

```

USING GIST ((geom::geography));

CREATE INDEX IF NOT EXISTS muc_tactile_polygons_geog_gix
ON muc_tactile_polygons_tmp_clean
USING GIST ((geom::geography));

```

After that:

```

ANALYZE muc_ways_noded_4326_v2;
ANALYZE muc_tactile_points_tmp_clean;
ANALYZE muc_tactile_lines_tmp_clean;
ANALYZE muc_tactile_polygons_tmp_clean;

```

ST_DWithin is designed for distance queries and can use spatial indexes.

Now, instead of recalculating every row through CASE, first reset the value:

```

UPDATE muc_ways_noded_4326_v2
SET tactile_weight = 0.0
WHERE tactile_weight IS DISTINCT FROM 0.0;

```

IS DISTINCT FROM is useful here: PostgreSQL updates only rows where the value actually differs from 0, instead of rewriting the entire table unnecessarily.

Then process points separately:

```

UPDATE muc_ways_noded_4326_v2 w
SET tactile_weight = 1.0
WHERE tactile_weight = 0.0
AND EXISTS (
    SELECT 1
    FROM muc_tactile_points_tmp_clean p
    WHERE ST_DWithin(
        w.geom::geography,
        p.geom::geography,
        5
    )
);

```

Then lines:

```

UPDATE muc_ways_noded_4326_v2 w
SET tactile_weight = 1.0
WHERE tactile_weight = 0.0
AND EXISTS (
    SELECT 1
    FROM muc_tactile_lines_tmp_clean l
    WHERE ST_DWithin(
        w.geom::geography,
        l.geom::geography,
        1
    )
);

```

And polygons:

```
UPDATE muc_ways_noded_4326_v2 w
SET tactile_weight = 1.0
WHERE tactile_weight = 0.0
AND EXISTS (
  SELECT 1
  FROM muc_tactile_polygons_tmp_clean a
  WHERE ST_DWithin(
    w.geom::geography,
    a.geom::geography,
    10
  )
);
```

5. Always Check the Result

After the UPDATE :

```
SELECT
  COUNT(*) AS total_edges,
  COUNT(*) FILTER (WHERE tactile_weight = 1.0) AS tactile_edges,
  ROUND(
    100.0 *
    COUNT(*) FILTER (WHERE tactile_weight = 1.0) /
    NULLIF(COUNT(*), 0),
    2
  ) AS tactile_percentage
FROM munich_ways;
```

This immediately shows whether something suspicious has happened, for example:

If 90% of the city has tactile_weight = 1, that almost certainly indicates an excessively large buffer or a geometry problem.

Before deleting the temporary tables, check separately how many routing edges were matched by each source.

Points:

```
SELECT COUNT(DISTINCT w.gid)
FROM munich_ways w
JOIN munich_tactile_points p
ON ST_DWithin(
  w.the_geom::geography,
  p.geom::geography,
  10
);
```

Lines:

```
SELECT COUNT(DISTINCT w.gid)
```

```
FROM munich_ways w
JOIN munich_tactile_lines l
  ON ST_DWithin(
    w.the_geom::geography,
    l.geom::geography,
    10
  );
```

Polygons:

```
SELECT COUNT(DISTINCT w.gid)
FROM munich_ways w
JOIN munich_tactile_polygons a
  ON ST_DWithin(
    w.the_geom::geography,
    a.geom::geography,
    10
  );
```

This immediately shows which geometry type actually matters in the OSM data for your city.

6. Delete the Temporary Tables After Verification

After confirming that the result is correct:

```
DROP TABLE IF EXISTS munich_tactile_points;
DROP TABLE IF EXISTS munich_tactile_lines;
DROP TABLE IF EXISTS munich_tactile_polygons;
```

And:

```
ANALYZE munich_ways;
```

17. SQL Patterns for External Sources

In the English-language QGIS interface, use [QuickOSM](#) and download only `node["highway"="street_lamp"]` so that the result contains actual street-lamp points.

First install the plugin if it is not already installed:

Plugins → Manage and Install Plugins...

In the search box, enter: QuickOSM

Click: Install Plugin

Then open the tool itself:

Vector → QuickOSM → QuickOSM

In the QuickOSM window, open the Quick query tab.

Fill it in as follows:

Key: highway

Value: street_lamp

In: München, Germany

Then click: Run query

If München, Germany does not work or produces an area that is too broad or unusual, use the more reliable Overpass query option.

Paste the following:

```
[out:xml][timeout:240];
area ["boundary"="administrative"] ["name"="München"] ["admin_level"="6"] -
>.searchArea;
( /* Individual mapped street lamps */ node["highway"="street_lamp";
/* Ways where lighting is mapped directly */ way["highway"]"lit";
/* Additional mapped light sources, useful for QA, not always street lamps */
node"light_source"; node"tower:type"="lighting"; way"tower:type"="lighting"; );
out body;
; out skel qt;
```

Then click: Run query

After it runs, a new layer should appear in the Layers panel.

Check the result: Right click on layer → Open Attribute Table

It should contain the field: highway with the value: street_lamp

Now save the layer to a file:

Right click on OsmQuery → Export → Save Features As...

In the export dialog, set:

Format: GeoPackage

File name: C:\Users\userrr\Documents\CAT\muc_streetlights_tmp.gpkg

Layer name: muc_streetlights_tmp

CRS: EPSG:4326 - WGS 84

Then click: OK

If you plan to upload this to Supabase later, it is better to keep only the minimum required fields before export. For the first pass, however, you can save it as is. The important part is that the result contains:

```
geometry Point/line EPSG:4326
osm_id
osm_type
highway = street_lamp
```

After uploading to Supabase, check:

```
select count(*) from public.muc_streetlights_tmp;

select
  st_srid(geom) as srid,
  geometrytype(geom) as geom_type,
  count(*)
from public.muc_streetlights_tmp
group by st_srid(geom), geometrytype(geom);
```

Expected result:

```
SRID = 4326
Geometry type = POINT
```

Spatial Update for light_weight

Because the database stores EPSG:4326, you cannot use a distance of 15 directly on geometry — that would be interpreted in degrees, not meters. There is no need to permanently reproject the tables. Use a cast from geometry to geography:

`geom::geography`

The table remains `geometry(..., 4326)`; the coordinates are not rewritten and the SRID does not change. Only the relevant PostGIS functions begin calculating distances in meters.

1. If a network edge inherits an OSM way with `lit=yes` / `lit=dusk-dawn` / `lit=automatic` / `lit=24/7` → `light_weight = 1`
2. If `lit=no` / `lit=disused` → `light_weight = 0`
3. If `lit=limited` / `lit=interval` → this is ambiguous: either use 0 or a separate confidence value; for binary `light_weight`, initially use 0 or review these cases separately
4. If `lit` is missing → use `street_lamp` coverage based on buffers
5. If there is neither a `lit` tag nor a nearby `street_lamp` → `light_weight = 0`, but this means “lighting not confirmed,” not “definitely dark”

Use a coverage-based approach: not “one nearby lamp means the whole street is lit,” but “what share of the edge line is covered by lamp buffers.” This prevents long streets from incorrectly receiving `light_weight = 1` because of a single lamp.

What to Upload to Supabase

Do not upload raw QuickOSM tables with hundreds of columns. Create two small staging tables instead.

Step 1. Rename the Raw Layers in QGIS

In the Layers panel, find the layer containing street-lamp points. Right-click the layer:
Rename Layer

Name it: raw_osm_lighting_points

Then find the line layer containing roads/paths with lit=*. Right-click: Rename Layer

Name it: raw_osm_lit_highways

How to check that these are the correct layers:

Right click layer → Properties → Information

For street lamps, it should be:

Geometry: Point

For lit highways, it should be Geometry: LineString or Geometry:

MultiLineString. For now, simply uncheck the Polygon / MultiPolygon layer.

Step 2. Run Automatic Cleanup Through the Python Console

Open Plugins → Python Console. Then click: Show Editor

Open the muc_light_prep.py script, configure the output paths, and click the green Run button.

After the script runs, two clean layers will appear in QGIS:

muc_street_lamps_tmp_clean

muc_lit_highway_ways_tmp_clean

The files will also be saved here:

```
C:\Users\userrr\Documents\CAT\muc_street_lamps_tmp_clean.gpkg  
C:\Users\userrr\Documents\CAT\muc_lit_highway_ways_tmp_clean.gpkg  
C:\Users\userrr\Documents\CAT\muc_lit_highway_ways_tmp_clean.csv
```

Step 3. Check the Result in QGIS

For muc_street_lamps_tmp_clean, open:

Right click → Open Attribute Table

It should contain only these fields:

lamp_id
full_id
osm_id
osm_type
source
geom/geometry.

For muc_lit_highway_ways_tmp_clean, it should contain only:

way_id
full_id
osm_id
osm_type
highway
lit
source
geom/geometry.

If everything is correct, these layers can already be uploaded to Supabase. They should not fail because of fields such as brewery:pils, because all raw OSM columns have been removed.

Step 4. Upload the Clean Layers to Supabase Through QGIS

The easiest method, if you already have a PostGIS connection to Supabase in QGIS, is:

Database → DB Manager...

Then: PostGIS → your Supabase connection → public

Then click: Import Layer/File...

For street lamps:

Input: muc_street_lamps_tmp_clean
Schema: public
Table: muc_street_lamps_tmp
Geometry column: geom
Source SRID: 4326
Target SRID: 4326
Primary key: lamp_id
For lit highways:

Input: muc_lit_highway_ways_tmp_clean
Schema: public
Table: muc_lit_highway_ways_tmp
Geometry column: geom
Source SRID: 4326
Target SRID: 4326
Primary key: way_id

Step 6. Checks After Upload

In the Supabase SQL Editor, run:

```
select count(*) from public.muc_street_lamps_tmp;

select
  st_srid(geom) as srid,
  geometrytype(geom) as geom_type,
  count(*)
from public.muc_street_lamps_tmp
group by st_srid(geom), geometrytype(geom);
```

Expected:

```
srid = 4326
geom_type = POINT
```

For lit highways:

```
select count(*) from public.muc_lit_highway_ways_tmp;

select
  lit,
  count(*)
from public.muc_lit_highway_ways_tmp
group by lit
order by count(*) desc;
```

Also check how many records match the network by osm_id:

```
select count(distinct w.gid) as matched_edges_by_osm_id
from public.muc_ways_noded_4326_v2 w
join public.muc_lit_highway_ways_tmp l
  on w.osm_id = l.osm_id
where w.osm_id is not null
  and l.osm_id is not null;
```

If this returns 0, check the osm_id / full_id format, although there should usually be matches. If QGIS reports that the table already exists, it may be easier not to create the tables manually before import. Let QGIS create them, then add the indexes in the Supabase SQL Editor. However, a predefined schema gives you direct control over the column types.

SQL for Debug Calculation Before the Update

Create the spatial indexes

This is safe to run even if some of the indexes already exist:

```
create index if not exists sidx_muc_street_lamps_tmp_geom on
public.muc_street_lamps_tmp using gist (geom);

create index if not exists sidx_muc_street_lamps_tmp_geog on
public.muc_street_lamps_tmp using gist ((geom::geography));

create index if not exists sidx_muc_ways_noded_4326_v2_geog on
public.muc_ways_noded_4326_v2 using gist ((geom::geography)) where geom is not
null;

create index if not exists idx_muc_lit_highway_ways_tmp_osm on
public.muc_lit_highway_ways_tmp (osm_id, osm_type);

create index if not exists idx_muc_lit_highway_ways_tmp_full_id on
public.muc_lit_highway_ways_tmp (full_id);

analyze public.muc_street_lamps_tmp; analyze public.muc_lit_highway_ways_tmp;
analyze public.muc_ways_noded_4326_v2; Step 3. Create the debug table
```

Use this SQL:

```
drop table if exists public.muc_light_decision_debug;
create table public.muc_light_decision_debug as with params as ( select
18.0::double precision as lamp_radius_m, 0.55::double precision as
min_coverage_ratio ),
osm_lit_by_edge as ( select w.gid, lower(trim(l.lit)) as lit_value from
public.muc_ways_noded_4326_v2 w join public.muc_lit_highway_ways_tmp l on
w.full_id is not null and l.full_id is not null and w.full_id = l.full_id
union all
select w.gid, lower(trim(l.lit)) as lit_value from public.muc_ways_noded_4326_v2 w
join public.muc_lit_highway_ways_tmp l on w.osm_id is not null and l.osm_id is not
null and w.osm_id = l.osm_id and coalesce(lower(w.osm_type), 'way') in ('way',
'w') ),
edge_base as ( select w.gid, w.geom,
max(
case
when l.lit_value in ('yes', 'automatic', 'dusk-dawn', '24/7', 'always')
then 1
else 0
end
) as has_positive_lit_tag,
max(
case
when l.lit_value in ('no', 'disused')
then 1
```

```

        else 0
    end
) as has_negative_lit_tag,

string_agg(distinct l.lit_value, ', ') as lit_values

from public.muc_ways_noded_4326_v2 w left join osm_lit_by_edge l on w.gid = l.gid
where w.geom is not null group by w.gid, w.geom ),
lamp_buffers as ( select lamp_id, geom::geography as lamp_geog, st_buffer(
geom::geography, (select lamp_radius_m from params), 'quad_segs=4' )::geometry as
lamp_buffer_geom from public.muc_street_lamps_tmp where geom is not null ),
edge_lamp_buffers as ( select e.gid, count(distinct l.lamp_id) as lamp_count,
st_unaryunion(st_collect(l.lamp_buffer_geom))::geography as lamps_buffer_geog from
edge_base e join lamp_buffers l on e.has_positive_lit_tag = 0 and
e.has_negative_lit_tag = 0 and st_dwithin( e.geom::geography, l.lamp_geog, (select
lamp_radius_m from params) ) group by e.gid ),
coverage as ( select e.gid, e.has_positive_lit_tag, e.has_negative_lit_tag,
e.lit_values, st_length(e.geom::geography) as edge_length_m,
coalesce(b.lamp_count, 0) as lamp_count, case when b.lamps_buffer_geog is null
then 0.0 else st_length( st_intersection( e.geom::geography, b.lamps_buffer_geog )
) end as covered_length_m from edge_base e left join edge_lamp_buffers b on e.gid
= b.gid )
select gid, edge_length_m, covered_length_m, lamp_count,
case when edge_length_m > 0 then covered_length_m / edge_length_m else 0.0 end as
coverage_ratio,
lit_values,
case when has_negative_lit_tag = 1 then 0 when has_positive_lit_tag = 1 then 1
when edge_length_m > 0 and covered_length_m / edge_length_m >= (select
min_coverage_ratio from params) then 1 else 0 end::smallint as light_weight_new,

case when has_negative_lit_tag = 1 then 'osm_lit_no' when has_positive_lit_tag = 1
then 'osm_lit_positive' when edge_length_m > 0 and covered_length_m /
edge_length_m >= (select min_coverage_ratio from params) then
'street_lamp_coverage' else 'unknown_or_not_covered' end as light_source
from coverage;
create index if not exists idx_muc_light_decision_debug_gid on
public.muc_light_decision_debug (gid);
analyze public.muc_light_decision_debug;

```

It is normal for this query to take a long time because it calculates spatial coverage. If it times out, split the calculation into several temporary/debug tables.

[Check the Summary](#)

First, check the overall result:

```

select count() as total_edges, count() filter (where light_weight_new = 1) as
lit_edges, count() filter (where light_weight_new = 0) as unknown_or_unlit_edges,
round( 100.0 * count() filter (where light_weight_new = 1) / nullif(count(*), 0),
2 ) as lit_percent from public.muc_light_decision_debug;

```

Then check the result by source:

```
select light_source, light_weight_new, count(*) as edge_count,
round(avg(coverage_ratio)::numeric, 3) as avg_coverage_ratio,
round(avg(edge_length_m)::numeric, 1) as avg_edge_length_m,
round(avg(lamp_count)::numeric, 1) as avg_lamp_count from
public.muc_light_decision_debug group by light_source, light_weight_new order by
light_source, light_weight_new;
```

Also review the coverage distribution separately:

```
select percentile_cont(0.1) within group (order by coverage_ratio) as p10,
percentile_cont(0.25) within group (order by coverage_ratio) as p25,
percentile_cont(0.5) within group (order by coverage_ratio) as median,
percentile_cont(0.75) within group (order by coverage_ratio) as p75,
percentile_cont(0.9) within group (order by coverage_ratio) as p90 from
public.muc_light_decision_debug where lamp_count > 0;
```

If lit_percent looks too low, for example 5–10%, you may need to increase lamp_radius_m to 20–25 or reduce min_coverage_ratio to 0.45.

If lit_percent is too high, for example 80–95%, the radius/threshold is too permissive: reduce the radius to 12–15 or increase the threshold to 0.65.

First, check that the debug table covers all network edges with geometry:

```
select
  (select count(*) from public.muc_ways_noded_4326_v2 where geom is not null) as
network_edges_with_geom,
  (select count(*) from public.muc_light_decision_debug) as debug_edges;
```

The values should match. The expected count is approximately 323184.

Then check how many edges have nearby lamps but coverage below the threshold:

```
select
  count(*) as edges_with_lamps_but_not_lit,
  round(avg(coverage_ratio)::numeric, 3) as avg_coverage_ratio,
  round(avg(edge_length_m)::numeric, 1) as avg_edge_length_m,
  round(avg(lamp_count)::numeric, 1) as avg_lamp_count
from public.muc_light_decision_debug
where lamp_count > 0
  and light_weight_new = 0
  and light_source = 'unknown_or_not_covered';
```

If there are only a few such objects, that is fine. If there are many, visually check whether the 0.55 threshold is too strict.

Also check long edges that became lit through street-lamp coverage:

```
select
  gid,
  edge_length_m,
  covered_length_m,
  coverage_ratio,
  lamp_count
```

```

from public.muc_light_decision_debug
where light_source = 'street_lamp_coverage'
order by edge_length_m desc
limit 50;

```

If there are no unusual 300–1000 meter segments with only one lamp, the coverage logic is working correctly.

Also check unusual or rare lit_values:

```

select
  lit_values,
  light_source,
  light_weight_new,
  count(*) as edge_count
from public.muc_light_decision_debug
where lit_values is not null
group by lit_values, light_source, light_weight_new
order by edge_count desc
limit 50;

```

This helps identify values such as limited, interval, yes;no, street_lamp, sunset-sunrise, and so on. If many valid “almost positive” values are currently being classified as unknown_or_not_covered, the list of positive lit values can be expanded.

Create a Backup Before the Update

Do not rely on an interactive `BEGIN ... COMMIT` sequence in the Supabase UI. It is more reliable to create a backup table, perform the update, and roll back with a separate SQL statement if necessary.

```

drop table if exists public.muc_light_weight_backup_before_20260810;

create table public.muc_light_weight_backup_before_20260810 as select gid,
light_weight from public.muc_ways_noded_4326_v2;

create unique index idx_muc_light_weight_backup_before_20260810_gid on
public.muc_light_weight_backup_before_20260810 (gid);

```

Update the Production Network

Only do this if the debug summary looks correct:

```

update public.muc_ways_noded_4326_v2 as w set light_weight = d.light_weight_new
from public.muc_light_decision_debug d where w.gid = d.gid;

```

Check production:

```

select count() as total_edges, count() filter (where light_weight = 1) as
lit_edges, count() filter (where light_weight = 0) as unknown_or_unlit_edges,
round( 100.0 * count() filter (where light_weight = 1) / nullif(count(*), 0), 2 )
as lit_percent from public.muc_ways_noded_4326_v2; If the result is incorrect,
roll back with: update public.muc_ways_noded_4326_v2 as w set light_weight =

```

```
b.light_weight from public.muc_light_weight_backup_before_20260810 b where w.gid = b.gid; Recommended Settings for the First Run
```

For the first production-like debug run:

```
lamp_radius_m = 18 min_coverage_ratio = 0.55
```

These settings are fairly conservative. You can then recreate `muc_light_decision_debug` with different parameters and compare the summaries before the update.

Then Remove the Street-Lamp Layer from Supabase

After `light_weight` has been populated and the result has been verified:

```
drop table if exists public.muc_streetlights_tmp;
```

Do not delete the backup immediately. Keep `muc_light_weight_backup_20260810` until you have confirmed that routing works correctly.

If you need to roll back `light_weight`:

```
update public.muc_ways_noded_4326_v2 as ways
set light_weight = backup.light_weight
from public.muc_light_weight_backup_20260810 as backup
where ways.gid = backup.gid;
```

Separate Layer for Visualization

After this, prepare a proper street-lamp layer locally for the application as a separate dataset. Do not mix it with the routing network.

For the visualization layer, the following is sufficient:

```
lamp_id
osm_id
osm_type
city
country
source
lon
lat
geometry
```

It can be stored as a separate `GeoJSON` information layer rather than as part of the routing table.

POI/amenities

For Munich, the goal is to reproduce the same structure currently used for Hamburg as closely as possible: first obtain six POI layers from OSM, save the GeoJSON files in `public/data/POI`, then upload the same layers to Supabase and use them to populate a binary weight in the road network.

1. First Define the Layer Structure

For Munich, use the following names:

Category	Supabase table	GeoJSON
Gastronomy	<code>poi_muc_gastronomy</code>	<code>public/data/POI/poi_muc_gastronomy.geojson</code>
Hospitals & Pharmacies	<code>poi_muc_health</code>	<code>public/data/POI/poi_muc_health.geojson</code>
Childcare & Schools	<code>poi_muc_kita_schule</code>	<code>public/data/POI/poi_muc_kita_schule.geojson</code>
Universities & Universities of Applied Sciences	<code>poi_muc_uni_fh</code>	<code>public/data/POI/poi_muc_uni_fh.geojson</code>
Parks & Playgrounds	<code>poi_muc_park_spiel</code>	<code>public/data/POI/poi_muc_park_spiel.geojson</code>
Supermarkets	<code>poi_muc_supermarket</code>	<code>public/data/POI/poi_muc_supermarket.geojson</code>

The OSM tags for these categories can be mapped as follows: Gastronomy — `restaurant`, `cafe`, `fast_food`, `pub`, `bar`, `biergarten`, `food_court`, `ice_cream`; Health — `hospital`, `pharmacy`; Education — `school`, `kindergarten`, `childcare`, `university`, `college`; Parks — `leisure=park|playground`; Supermarkets — `shop=supermarket`. This matches the current OSM tagging model.

If strict 1:1 compatibility with Hamburg is required, open the existing `poi_hh_*.geojson` files in QGIS before export and inspect the unique values in the `amenity`, `shop`, and `leisure` fields. This will show, for example, whether `childcare`, `pub`, `ice_cream`, and similar tags were included in Hamburg. Otherwise, the classification below is a reasonable OSM-based mapping, but it may differ from the older Hamburg export by a few tags.

2. Retrieve the Data Through Overpass Turbo

Open `Overpass Turbo` via `QuickOSM` and run one query for each layer.

{{geocodeArea:...}} lets you select the city's administrative area through Nominatim, while out center reduces OSM ways/relations to a single point, which is convenient specifically for POI layers.

Gastronomie:

```
[out:json][timeout:180];

{{geocodeArea:"München, Bayern, Deutschland"}}->.a;

nwr["amenity"~"^(restaurant|cafe|fast_food|pub|bar|biergarten|food_court|ice_cream)"](area.a);

out geom;
```

Krankenhäuser & Apotheken:

```
[out:json][timeout:400];
area(3600062428)->.a;
( nwr"amenity"="hospital"; nwr"amenity"="pharmacy");
out geom qt;
Kita & Schule:
[out:json][timeout:400];

area(3600062428)->.a;

(
nwr["amenity"="kindergarten"](area.a);
nwr["amenity"="childcare"](area.a);
nwr["amenity"="school"](area.a);
)
out geom qt;
```

Uni & FH:

```
[out:json][timeout:400];

area(3600062428)->.a;
(
nwr["amenity"="university"](area.a);
nwr["amenity"="college"](area.a);
)

out geom qt;
```

Parks & Spielplätze:

```
[out:json][timeout:400];

area(3600062428)->.a;

nwr["leisure"~"^(park|playground)"](area.a);
```

```
out geom qt;  
Supermärkte:
```

```
[out:json][timeout:400];  
  
area(3600062428)->.a;  
  
nwr["shop"="supermarket"](area.a);  
  
out geom qt;
```

After each query: Run → visually inspect the result → Export → GeoJSON.

Overpass Turbo exports the result as a GeoJSON FeatureCollection and keeps the object's OSM tags in properties.

Save the files as follows:

```
public/data/POI/poi_muc_gastronomy.geojson  
public/data/POI/poi_muc_health.geojson  
public/data/POI/poi_muc_kita_schule.geojson  
public/data/POI/poi_muc_uni_fh.geojson  
public/data/POI/poi_muc_park_spiel.geojson  
public/data/POI/poi_muc_supermarket.geojson
```

3. Check the GeoJSON Files in QGIS

Before uploading to Supabase, load all six files into QGIS and check: CRS = EPSG:4326; geometries are not empty; all points are actually inside München; there are no obvious POIs from Landkreis München; and the fields name, amenity, shop, leisure, and @id have been preserved.

4. Prepare the Table for QGIS

There is no benefit in uploading six full OSM layers with all QuickOSM fields to Supabase, because this would consume a large amount of storage.

Install once through PowerShell:

```
py -m pip install geopandas pyogrio
```

Open clean_munich_poi.py in the QGIS Python Console. Only change the local path to the folder containing all of the POI GeoJSON files:

```
BASE_DIR = Path(  
    r"C:\path\to\your\project\public\data\POI"
```

)

In the QGIS Python Editor, press Ctrl + Shift + E or click Run Script.

After the `exec(...)` line, the console should show something like:

```
Reading: poi_muc_gastronomy.geojson
Features: 1234
Columns before: 47
Empty columns removed: 28
Columns after: 19
Reading: poi_muc_health.geojson
...
```

At the very end:

```
Building minimal Supabase layer...
Done.
Supabase features: ...
Duplicate OSM objects collapsed: ...
Supabase file: C:\Users\ ...
```

5. Work with the `poi_muc_amenities_supabase` Table and Configure `facilities_weight`

First, add `node_id` directly to the already uploaded POI layer:

```
ALTER TABLE public.poi_muc_amenities_supabase
ADD COLUMN IF NOT EXISTS node_id bigint;
```

First run a check on 20 POIs without changing anything in the database:

```
SELECT
    p.gid,
    p.name,
    nearest.edge_gid,
    nearest.node_id,
    ROUND(nearest.snap_distance_m::numeric, 1) AS snap_distance_m
FROM public.poi_muc_amenities_supabase p

CROSS JOIN LATERAL (
    SELECT
        w.gid AS edge_gid,

        CASE
            WHEN ST_Distance(
                p.geom::geography,
                ST_StartPoint(w.geom)::geography
            )
            <=
            ST_Distance(
                p.geom::geography,
                ST_EndPoint(w.geom)::geography
            )
            THEN w.source
            ELSE w.target
        END AS node_id,
```

```

        LEAST(
            ST_Distance(
                p.geom::geography,
                ST_StartPoint(w.geom)::geography
            ),
            ST_Distance(
                p.geom::geography,
                ST_EndPoint(w.geom)::geography
            )
        ) AS snap_distance_m

FROM public.muc_ways_noded_4326_v2 w

WHERE w.geom IS NOT NULL
      AND w.source IS NOT NULL
      AND w.target IS NOT NULL

ORDER BY w.geom <-> p.geom
LIMIT 1
) nearest

WHERE p.geom IS NOT NULL
LIMIT 20;

```

The result will look approximately like this:

gid	name	edge_gid	node_id	snap_distance_m
1	REWE	12345	78192	8.4
2	Apotheke	87123	19283	14.1
3	Restaurant	98312	28173	5.7

Pay particular attention to `snap_distance_m`. If it is usually in the tens of meters, the logic looks reasonable. If you see `500`, `1000`, or `3000` meters, stop and investigate why.

Then I connect each point of interest (POI) to the nearest node in the road network:

```

UPDATE public.poi_muc_amenities_supabase p
SET node_id = (
    SELECT
        CASE
            WHEN ST_Distance(
                p.geom::geography,
                ST_StartPoint(w.geom)::geography
            )
            <=
            ST_Distance(
                p.geom::geography,
                ST_EndPoint(w.geom)::geography
            )
        THEN w.node_id
        ELSE NULL
    END
FROM public.muc_ways_noded_4326_v2 w
)

```

```

        THEN w.source
        ELSE w.target
    END
FROM public.muc_ways_noded_4326_v2 w
WHERE w.geom IS NOT NULL
    AND w.source IS NOT NULL
    AND w.target IS NOT NULL
ORDER BY w.geom <-> p.geom
LIMIT 1
)
WHERE p.geom IS NOT NULL
    AND p.node_id IS NULL;

```

If Supabase terminates the query after only some objects have been processed, rerunning it will not recalculate POIs that have already been filled.

After it finishes:

```

SELECT
    COUNT(*) AS total_poi,
    COUNT(node_id) AS with_node_id,
    COUNT(*) - COUNT(node_id) AS missing_node_id
FROM public.poi_muc_amenities_supabase;

```

Ideally, the result should be approximately:

total_poi	with_node_id	missing_node_id
7269	7269	0

Then:

```

CREATE INDEX IF NOT EXISTS idx_poi_muc_amenities_node_id
ON public.poi_muc_amenities_supabase(node_id);

```

At this point, the first stage is complete and the result has already been physically stored in Supabase.

Second stage:

Before that, check that cost is actually measured in meters:

```

SELECT
    gid,
    cost,
    ST_Length(geom::geography) AS length_m
FROM public.muc_ways_noded_4326_v2
WHERE geom IS NOT NULL
    AND cost IS NOT NULL
LIMIT 20;

```

If the result is approximately:

cost	length_m
42.1	42.1
87.6	87.6
15.3	15.3

then everything is correct: 300 really corresponds to approximately 300 meters.

```
DROP TABLE IF EXISTS public.tmp_muc_poi_dist;

CREATE TABLE public.tmp_muc_poi_dist AS
SELECT *
FROM pgr_drivingDistance(
    ,
    SELECT
        gid AS id,
        source,
        target,
        cost
    FROM public.muc_ways_noded_4326_v2
    WHERE source IS NOT NULL
        AND target IS NOT NULL
        AND cost IS NOT NULL
    ,
    ARRAY(
        SELECT DISTINCT node_id
        FROM public.poi_muc_amenities_supabase
        WHERE node_id IS NOT NULL
    ),
    300,
    false
);
```

Only then proceed to the third stage — the weight.

It is important to separate two things here: the column DEFAULT and the actually calculated weight value.

According to the documentation, facilities_weight is Type B, meaning it is a positive factor:

1 = a facility/amenity is available nearby,
0 = it is absent.

At the same time, the schema recommends DEFAULT 1.0 for positive factors so routing continues to work neutrally even if the factor has not yet been calculated at all.

However, once facilities_weight is actually calculated from POI data, better use a different logic: if the factor is found, use 1; otherwise use 0. This is shown by the light_weight example: THEN 1.0 ELSE 0.0.

So this: DEFAULT 1 does not mean that “all roads without amenities should remain 1.” Instead, it means: until the factor has been calculated, keep a safe neutral value. So,

reset the values to 0 before assigning 1 was correct. However, it is better not to do this directly in the current production column.

A better approach is to set up a separate column before rewriting the final one:

```
ALTER TABLE public.muc_ways_noded_4326_v2
ADD COLUMN IF NOT EXISTS facilities_weight_new smallint NOT NULL DEFAULT 0;
```

For the temporary calculation column, DEFAULT 0 is logical because it is being built from scratch:

initially: no amenities found → 0

availability found within 300 m → 1

Then:

```
UPDATE public.muc_ways_noded_4326_v2 w
SET facilities_weight_new = 1
FROM public.tmp_muc_poi_dist d
WHERE w.source = d.node;
```

and:

```
UPDATE public.muc_ways_noded_4326_v2 w
SET facilities_weight_new = 1
FROM public.tmp_muc_poi_dist d
WHERE w.target = d.node;
```

After that, check:

```
SELECT
    facilities_weight_new,
    COUNT(*)
FROM public.muc_ways_noded_4326_v2
GROUP BY facilities_weight_new
ORDER BY facilities_weight_new;
```

Only when the result looks correct:

```
UPDATE public.muc_ways_noded_4326_v2
SET facilities_weight = facilities_weight_new;
```

Then remove the calculation column:

```
ALTER TABLE public.muc_ways_noded_4326_v2
DROP COLUMN IF EXISTS facilities_weight_new;
```

18. Process WMS and Rasters Separately

Noise

In the source data, noise is available as WMS through GetFeatureInfo, with a threshold above 65 dB.

WMS cannot be cleanly loaded into PostGIS as a GeoJSON table. A better approach is:

muc_ways_final with gid

- create points along each line in QGIS
- retrieve noise values
- aggregate the maximum by gid
- noise_weight = 1 if maximum >= 65
- export a table with gid + noise_weight

Then upload the table as: staging.muc_noise_edge_flags

And run:

```
UPDATE staging.muc_ways_work AS w
SET noise_weight = flags.noise_weight
FROM staging.muc_noise_edge_flags AS flags
WHERE flags.gid = w.gid;
```

This is better than storing the entire noise raster in Supabase.

Given that the road network already exists in Supabase with stable gid values, use the simplest and safest approach:

perform all spatial processing locally in QGIS, upload only a small gid → mean_dB table to Supabase, and update noise_weight with a regular JOIN on gid.

Do not upload the 5-meter buffers themselves to Supabase and then perform another spatial join on the server. That would add unnecessary geometry, CRS complications, and heavier SQL queries without any benefit.

For WMS, QGIS receives noise data through GetFeatureInfo, so this is not an RGB image value. The source noise layer itself is a raster layer in EPSG:25832, and LfU states that the calculations were performed on a 10 × 10 m grid.

Keep the production geometry in Supabase in EPSG:4326.

Do not transform muc_ways_noded_4326_v2.geom to 25832. The backend already works with this table, and the documentation also assumes EPSG:4326 for the routing graph.

First, verify the actual SRID:

```
SELECT
  ST_SRID(geom) AS srid,
  ST_GeometryType(geom) AS geometry_type,
  COUNT(*) AS count
FROM public.muc_ways_noded_4326_v2
WHERE geom IS NOT NULL
GROUP BY ST_SRID(geom), ST_GeometryType(geom);
```

Expected:

srid		geometry_type		count
4326		ST_LineString/...		...

The table name contains 4326, but the geom geometry declaration itself does not restrict the SRID, so this check is important.

Load the roads from Supabase into QGIS and create a local working copy.

You only need:

gid
geom

In QGIS, connect to PostgreSQL/Supabase, open `muc_ways_noded_4326_v2`, then:

Right click layer → Export → Save Features As...

Format: GeoPackage

CRS: EPSG:25832 – ETRS89 / UTM zone 32N

For example: `muc_ways_noise_work.gpkg`

Keep only these fields:

gid
geom

It is critical to preserve the original gid.

In the screenshot, the QGIS project CRS in the lower-right corner is currently EPSG:4326. For the next processing steps, switch the Project CRS to EPSG:25832 as well to avoid any confusion about units.

The production database remains in 4326. Only the local working layer is converted to 25832.

Create a 5-meter buffer.

In Processing Toolbox, search for: Buffer

Parameters:

Input layer: `muc_ways_noise_work`

Distance: 5

Segments: 5

Dissolve result: NO

For example, name the result: `muc_ways_buffer_5m`

The structure will be:

gid	geometry
1234	POLYGON(...)
1235	POLYGON(...)
1236	POLYGON(...)

Dissolve = NO is essential: each gid needs its own separate buffer.

Because CRS 25832 uses meters:

5 = 5 metres

Create the points that will be used to sample the raster.

The source is not a downloaded numeric GeoTIFF but a WMS. Therefore, a literal Raster pixels to points operation may process the rendered WMS image rather than the Pegel in dB(A) values.

Instead, create an equivalent point sample. Because the original LfU raster has 10×10 m cells, create a regular point grid with the following spacing: $10 \text{ m} \times 10 \text{ m}$

Processing Toolbox: Create grid

Grid type: Point

Horizontal spacing: 10

Vertical spacing: 10

CRS: EPSG:25832

Extent — the area covered by the road network. You can use your screen extent.

Then: Extract by location

keep only the points that:

intersect

intersect muc_ways_buffer_5m.

This leaves only points near roads instead of a huge grid covering all of Munich.

For each point, retrieve Pegel in dB(A) from the WMS.

The file sample_lfu_noise_wms.py intentionally contains: `MAX_POINTS = 100`

This means it will initially process only 100 points. Keep it this way for the first test and verify that the table contains values such as 57.8, 66.2, and so on. If everything is correct, change it to: `MAX_POINTS = None`

Before running the script, two layers with these exact names must be loaded in QGIS:

noise_sample_points
Straßen gesamt LDEN 2022

It is better to save noise_sample_points as a local GeoPackage rather than a temporary layer.

Calculate the mean value for each 5 m buffer.

Processing Toolbox: Join attributes by location (summary)

Input layer: muc_ways_buffer_5m
Join layer: noise_sample_points
Predicate: intersects
Field: db_value

Summaries:

mean
count

The output will be:

gid	db_value_mean	db_value_count
1234	57.83	4
1235	68.17	6
1236	61.45	3

This is the final QGIS result.

Upload only the table to Supabase, without geometry.

Export the result as CSV with only three fields:

gid,noise_db_mean,noise_sample_count
1234,57.83,4
1235,68.17,6
1236,61.45,3

Do not send any of the following to the server:

buffers
polygons
EPSG:25832 geometry
noise points

They are not needed on the server.

Create a staging table:

```
CREATE TABLE public.muc_noise_lden_2022_tmp (  
  gid bigint PRIMARY KEY,  
  noise_db_mean double precision,  
  noise_sample_count integer  
);
```

The CSV can be imported directly through the Supabase Table Editor.

Add the raw noise value to the main table.

It is recommended to store the dB value next to noise_weight. This is very useful for QA and for changing the threshold later:

```
ALTER TABLE public.muc_ways_noded_4326_v2  
ADD COLUMN IF NOT EXISTS noise_db_mean double precision,  
ADD COLUMN IF NOT EXISTS noise_sample_count integer;
```

First reset the old values:

```
UPDATE public.muc_ways_noded_4326_v2  
SET  
  noise_weight = 0,  
  noise_db_mean = NULL,  
  noise_sample_count = NULL;
```

Then transfer the result using the exact gid, with no spatial operations at all:

```
UPDATE public.muc_ways_noded_4326_v2 AS w  
SET  
  noise_db_mean = n.noise_db_mean,  
  noise_sample_count = n.noise_sample_count,  
  noise_weight = CASE  
    WHEN n.noise_db_mean > 65 THEN 1  
    ELSE 0  
  END  
FROM public.muc_noise_lden_2022_tmp AS n  
WHERE w.gid = n.gid;
```

Note the > 65 condition. If you require “65 dB and above,” change to >= 65.

Check the result.

```
SELECT  
  noise_weight,  
  COUNT(*) AS edges  
FROM public.muc_ways_noded_4326_v2  
GROUP BY noise_weight  
ORDER BY noise_weight;
```

And:

```
SELECT  
  MIN(noise_db_mean) AS min_db,  
  AVG(noise_db_mean) AS avg_db,  
  MAX(noise_db_mean) AS max_db,  
  COUNT(noise_db_mean) AS edges_with_noise_data,
```

```
COUNT(*) - COUNT(noise_db_mean) AS edges_without_noise_data
FROM public.muc_ways_noded_4326_v2;
```

You can also check the noisiest links:

```
SELECT
    gid,
    noise_db_mean,
    noise_sample_count,
    noise_weight
FROM public.muc_ways_noded_4326_v2
WHERE noise_db_mean IS NOT NULL
ORDER BY noise_db_mean DESC
LIMIT 50;
```

After verification:

```
DROP TABLE public.muc_noise_lden_2022_tmp;
```

The production table then contains only:

gid	
geom	EPSG:4326

...

noise_db_mean	68.17
noise_sample_count	6
noise_weight	1

Architecturally, the flow is:

```
Supabase roads EPSG:4326
  ↓
local QGIS copy
  ↓
reproject EPSG:25832
  ↓
buffer 5 m by gid
  ↓
10×10 m sample points
  ↓
LfU GetFeatureInfo → dB(A)
  ↓
mean(dB) inside each buffer
  ↓
CSV gid → mean_dB
  ↓
Supabase
  ↓
JOIN ON gid
  ↓
mean_dB > 65
```

↓
noise_weight = 1

This is the simplest approach for this project. A spatial join in Supabase is not needed here. Geometry is used once locally, where it is convenient, and the server receives a ready result keyed by the existing primary key gid.

Traffic Lights

First connect to WFS directly instead of downloading WMS.

In QGIS with the English interface:

Layer → Add Layer → Add WFS / OGC API - Features Layer...

Alternatively:

Data Source Manager → WFS / OGC API - Features

Click New.

In Name, enter for example: Munich Traffic Lights

In URL, enter the base WFS endpoint:

https://geoportal.muenchen.de/geoserver/mor_wfs/ows (Munich example)

Use the endpoint itself, not the full GeoJSON URL with GetFeature.

Then: OK → Connect

A layer should appear with a name similar to:

mor_wfs:Lsa_opendata or Lsa_opendata

Select it → Add.

Local Check in QGIS

You should have two layers:

munich_ways

Lsa_opendata

First, check the CRS.

For munich_ways:

Right click → Properties → Information

Find: CRS

It should be: EPSG:4326 - WGS 84

Do the same for Lsa_opendata.

If the WFS is currently delivered in EPSG:25832, that is normal for the source. Do not click Set CRS or change the layer CRS manually — that does not transform the coordinates; it only changes their declaration.

For testing, create a temporary copy of the LSA layer in a metric CRS. This is only a temporary QGIS layer and will not be uploaded to Supabase.

Open: Processing → Toolbox

If the panel is not visible: View → Panels → Processing Toolbox

In the search box, enter: Reproject layer

Open Reproject layer.

Parameters:

Input layer:

Lsa_opendata

Target CRS: EPSG:25832 - ETRS89 / UTM zone 32N

Output: Temporary Layer

Click Run.

Why EPSG:25832 specifically: the official LSA WFS publishes its geometry in this CRS, and it is a metric UTM system for the Munich region. For distance analysis, it is much more appropriate than Web Mercator 3857.

Now temporarily do the same with the roads.

Again: Processing Toolbox → Reproject layer

Input: munich_ways

Target CRS: EPSG:25832

Output: Temporary Layer

Run.

You can rename the layers in the Layers panel: traffic_lights_25832, ways_25832

These are only local temporary layers.

Create a 20-Meter Buffer

In Processing Toolbox find Buffer

Open the regular Buffer tool.

Parameters:

Input layer: traffic_lights_25832

Distance: 20

Segments: 5

End cap style: Round

Join style: Round

Dissolve result: unchecked

Output: Temporary Layer

Run.

The result will be a layer, for example: Buffered

Rename it: lsa_buffer_20m

You will now literally see a 20 m radius around each LSA point.

Create 10 m and 30 m Buffers for Comparison

Run Buffer two more times.

First:

Distance: 10

name it: lsa_buffer_10m

Second:

Distance: 30

name it: lsa_buffer_30m

You now have three variants. In Layers, you can switch on in turn:

☒ ways_25832

☒ traffic_lights_25832

☒ lsa_buffer_10m

☐ lsa_buffer_20m

☐ lsa_buffer_30m

then 20, then 30.

This quickly shows which radius behaves reasonably.

How to Check Which Roads Intersect the 20-Meter Buffer

Use: Processing Toolbox → Extract by location

Parameters:

Extract features from: ways_25832

Where the features: intersect

By comparing to the features from: lsa_buffer_20m

Output: Temporary Layer

Run.

A new layer will be created.

Name it: ways_lsa_20m

This layer contains all road edges that intersect the 20-meter zones.

Do the same for:

lsa_buffer_10m → ways_lsa_10m

lsa_buffer_30m → ways_lsa_30m

You can now switch the following layers on and off:

ways_lsa_10m

ways_lsa_20m

ways_lsa_30m

and compare the difference.

How to Count the Selected Edges

The simplest method is: Right click ways_lsa_20m → Open Attribute Table

At the bottom, QGIS will show: Features total: ...

Compare:

10 m → X edges

20 m → Y edges

30 m → Z edges

For example, if the result is:

10 m → 1 850

20 m → 4 100

30 m → 9 600

it is already clear that 30 m starts to expand the selection sharply.

After Verification, Prepare the LSA Layer for Supabase

Now return to the original Lsa_opendata layer.

Create a permanent copy specifically in EPSG:4326.

Right click Lsa_opendata → Export → Save Features As...

In the dialog that opens:

Format:

GeoPackage

File name, for example:

`munich_lsa.gpkg`

Layer name:

`munich_traffic_lights`

CRS:

EPSG:4326 - WGS 84

This performs a real coordinate transformation during export.

The geometry will be saved as Point in EPSG:4326.

Click OK.

Then check again:

Right click exported layer → Properties → Information

It should be:

Geometry:

Point

CRS:

EPSG:4326

11. How to Upload This to Supabase

There are two methods.

For spatial data, it is preferable to connect QGIS directly to PostgreSQL/Supabase and export the layer there. Supabase supports CSV import through the dashboard, but CSV is inconvenient for PostGIS geometry. Supabase documents the SQL Editor/PostGIS workflow and several bulk-import methods, while spatial queries are handled through PostGIS.

First, in Supabase:

Project → Database

find the connection parameters / connection string.

You need:

Host

Port
Database
User
Password

In QGIS: Browser panel → PostgreSQL

If the Browser panel is not visible: View → Panels → Browser Panel

Right-click: PostgreSQL → New Connection

Enter the Supabase connection details.

Important: if a direct database connection from QGIS fails because of IPv6/network restrictions, use the Supabase Session Pooler connection string from the Dashboard instead of the direct connection.

After connecting, the Supabase PostgreSQL database should appear in the Browser panel.

Export the LSA Layer from QGIS Directly to Supabase

Right-click: munich_traffic_lights

Export → Save Features As...

Format: PostgreSQL

Connection: your Supabase connection

Schema: public

Table: munich_traffic_lights

Geometry column: geom

CRS: EPSG:4326

If the following option is available: Create spatial index enable it.

If the following option is available: Lower case field names enable it as well.

OK.

After this, the table will physically exist in PostgreSQL.

Check the Table in Supabase

Open: Supabase Dashboard → SQL Editor → New query

First:

```
SELECT COUNT(*)  
FROM public.munich_traffic_lights;
```

The count should be approximately the same as the number of features in QGIS.

Then:

```
SELECT  
  ST_GeometryType(geom) AS geometry_type,  
  ST_SRID(geom) AS srid,  
  COUNT(*) AS count  
FROM public.munich_traffic_lights  
GROUP BY  
  ST_GeometryType(geom),  
  ST_SRID(geom);
```

Expected:

```
ST_Point  
4326  
...
```

Then check a sample:

```
SELECT  
  standort,  
  ST_X(geom) AS lon,  
  ST_Y(geom) AS lat  
FROM public.munich_traffic_lights  
LIMIT 20;
```

For Munich, the coordinates should be approximately in the range:

```
longitude ≈ 11.x  
latitude ≈ 48.x
```

If you see values such as:

```
690000  
5330000
```

the data is still in UTM and the import was done incorrectly.

Check the Ways Before the Spatial Query

The example used a placeholder `w.id`, but in your table the edge identifier is called `gid`. That is why Postgres reports: `column w.id does not exist`

In `public.muc_ways_noded_4326_v2`, use: `w.gid`

The corrected query is therefore:

```
SELECT
    w.gid AS gid,
    l.standort
FROM public.muc_ways_noded_4326_v2 AS w
JOIN public.munich_traffic_lights AS l
    ON ST_DWithin(
        w.geom::geography,
        l.geom::geography,
        30
    )
LIMIT 100;
```

What it does: it takes edges from `muc_ways_noded_4326_v2`, takes traffic-light points from `munich_traffic_lights`, and returns pairs where the line is no more than 30 meters from the point. It does not modify anything in the database; this is only a check.

Proceed step by step.

First, check that the query works at all:

```
SELECT
    w.gid,
    l.standort
FROM public.muc_ways_noded_4326_v2 AS w
JOIN public.munich_traffic_lights AS l
    ON ST_DWithin(
        w.geom::geography,
        l.geom::geography,
        30
    )
LIMIT 100;
```

Then count how many unique edges fall within the 30-meter zone:

```
SELECT
    COUNT(DISTINCT w.gid) AS matched_ways
FROM public.muc_ways_noded_4326_v2 AS w
JOIN public.munich_traffic_lights AS l
    ON ST_DWithin(
        w.geom::geography,
```

```

        1.geom::geography,
        30
    );

```

For comparison, also check 20 meters:

```

SELECT
    COUNT(DISTINCT w.gid) AS matched_ways
FROM public.muc_ways_noded_4326_v2 AS w
JOIN public.munich_traffic_lights AS l
    ON ST_DWithin(
        w.geom::geography,
        l.geom::geography,
        20
    );

```

trafficlight_weight already exists, so there is no need to run ALTER TABLE. Once the 30 m radius has been confirmed, the final update will be:

```

UPDATE public.muc_ways_noded_4326_v2
SET trafficlight_weight = 0;
munich_traffic_lights does not have an index specifically on the geom::geography
expression.
muc_ways_noded_4326_v2 already has one. munich_traffic_lights only has a regular
geometry index. Therefore, first add a geography index for the traffic lights:
CREATE INDEX IF NOT EXISTS sidx_munich_traffic_lights_geog
ON public.munich_traffic_lights
USING GIST ((geom::geography));

```

and then:

```

UPDATE public.muc_ways_noded_4326_v2 AS w
SET trafficlight_weight = 1
FROM (
    SELECT DISTINCT matched.gid
    FROM public.munich_traffic_lights AS l
    CROSS JOIN LATERAL (
        SELECT w2.gid
        FROM public.muc_ways_noded_4326_v2 AS w2
        WHERE ST_DWithin(
            w2.geom::geography,
            l.geom::geography,
            30
        )
    ) AS matched
) AS affected
WHERE w.gid = affected.gid;

```

19. Final Check

The command consists of two operations:

VACUUM FULL → physically shrinks and rewrites the table

ANALYZE → updates query-planning statistics

Main Risk: the Table Will Be Locked

VACUUM FULL acquires an ACCESS EXCLUSIVE lock. While the command is running, other queries on this table may wait for the operation to finish. During execution, avoid running pgRouting calculations, opening this layer in QGIS, or accessing it from the backend.

The command also cannot run inside:

```
BEGIN;  
...  
COMMIT;
```

Run it as a separate SQL statement.

Temporary Increase in Size

PostgreSQL first creates a new copy of the table and indexes, and removes the old ones only after successful completion.

Current size:

```
table:    205 MB  
indexes:   61 MB  
total:    266 MB
```

At peak usage, the operation may temporarily require space roughly equal to the size of the rewritten table and its indexes, plus WAL and internal overhead. PostgreSQL explicitly warns that VACUUM FULL requires additional disk space until the rewrite is complete.

The Supabase Free Plan database limit is 500 MB, although the project's physical disk is larger than that limit. Even so, because of WAL and the current storage usage, there is no guarantee that the operation will complete without running out of space.

How to shrink the Table

This cannot be known in advance.

If a significant part of the 205 MB consists of:

- old row versions left by numerous UPDATE operations;
- deleted rows;
- recently dropped columns;
- bloated indexes,

the reduction may be significant.

If almost all of the 205 MB consists of the current 323 thousand geometries and live column values, the reduction may be only a few megabytes.

First, record the current sizes:

```
SELECT
  pg_size_pretty(
    pg_relation_size(
      'public.muc_ways_noded_4326_v2'
    )
  ) AS table_size,
  pg_size_pretty(
    pg_indexes_size(
      'public.muc_ways_noded_4326_v2'
    )
  ) AS indexes_size,
  pg_size_pretty(
    pg_total_relation_size(
      'public.muc_ways_noded_4326_v2'
    )
  ) AS total_size,
  pg_size_pretty(
    pg_database_size(current_database())
  ) AS database_size;
```

Then run separately:

```
VACUUM (FULL, ANALYZE)
public.muc_ways_noded_4326_v2;
```

After it completes, run the first query again and compare the values.